NEURAL COMPUTING AND DEEP LEARNING

Module: Neural Computing and Deep Learning

Eranga Kodikara

SID 2520175

Part A - Cassava Leaf Disease Classification

A.1. Dataset

I'm using the Kaggle Cassava Leaf Disease Classification dataset. It has 21,397 leaf photos across five classes. I ran the experiments inside a Kaggle notebook so I could attach the competition data directly without having to download 6 GB locally.

A quick summary of what I worked with:

Item	Value
Source	Kaggle competition: Cassava Leaf Disease Classification
Image size	600 × 800 RGB (uniform across all images)
Total images	21,397
Classes	5
Working subset	10,000 images (stratified)
Train / Val	8,000 / 2,000, 80/20 stratified

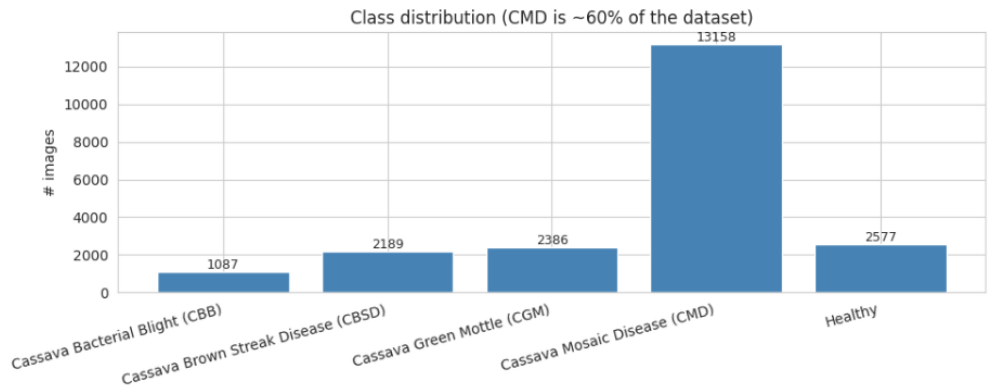
Class distribution

The dataset is heavily imbalanced. Cassava Mosaic Disease alone is over 60% of all images, and Cassava Bacterial Blight is only about 5%.

Label	Class	Count	%	Train share	Class weight
0	Cassava Bacterial Blight (CBB)	1,087	5.1%	406	3.941
1	Cassava Brown Streak Disease (CBSD)	2,189	10.2%	819	1.954
2	Cassava Green Mottle (CGM)	2,386	11.2%	892	1.794
3	Cassava Mosaic	13,158	61.5%	4,920	0.325

Label	Class	Count	%	Train share	Class weight
	Disease (CMD)				
4	Healthy	2,577	12.0%	963	1.661

A model that simply outputs “CMD” for every image would get 61.5% accuracy without learning anything, so I tracked Macro-F1 as the main metric throughout, not raw accuracy.




```
{0: 1087, 1: 2189, 2: 2386, 3: 13158, 4: 2577}
```

Figure 1. Class distribution bar chart from the notebook.

Figure 2. Four sample images per class (CBB, CBSD, CGM, CMD, Healthy).

When I looked at sample images per class side by side, what jumped out was how visually similar CMD and CGM are. Both produce green-toned mottling on the leaves.

A.2. Pre-processing pipeline

The data pipeline isn't where the marks are, but a few choices here made a real difference and I want to call them out.

Image size. I resized everything to 224 × 224 because that's the native input size for both VGG16 and MobileNetV2 with their ImageNet weights. If I'd picked a different size I'd have to add an extra resize layer inside the model graph, which adds parameters for no reason.

Pixel scaling. The from-scratch CNNs see pixel values divided by 255 so they're in [0, 1]. The pretrained backbones get their own ImageNet-style normalisation by calling applications.vgg16.preprocess_input (or the MobileNetV2 equivalent) inside the model graph. Mixing the two scaling regimes was a mistake I made on my first attempt and the VGG16 frozen baseline got terrible accuracy until I noticed it.

Augmentation. I used a **keras.Sequential** augmentation block with random horizontal+vertical flip, ±15° rotation, ±15% zoom and ±15% contrast. I picked these specific transformations because:

- A leaf doesn't have a "right way up" so flipping in any direction is fine.
- Rotation simulates the camera not being held perfectly straight.
- Zoom simulates closer or further-away photos.
- Contrast simulates different lighting (some images in the dataset are taken in bright sun, others in shade).

Crop and translation augmentations are more risky on this dataset because the diseased region might already be small, and cropping it out by accident would teach the model to predict "healthy" when the lesion just isn't visible.

Figure 3. A 2 × 6 grid of augmented training samples. Same images keep getting transformed differently each epoch, so the network effectively sees approximately 5× more training data than the raw 8k images.

Class weights instead of oversampling. This is the choice I'm most confident about. I computed sklearn's balanced class weights from the training subset only, which gave: CBB → 3.941, CBSD → 1.954, CGM → 1.794, CMD → 0.325, Healthy → 1.661. Oversampling would have meant duplicating rare-class images, and any duplicate that happened to land in both train and val would have inflated my val score. Weighting just adjusts the loss without touching the data.

Stratified train/val split with seed 42. Without stratification, the rare classes have unstable F1 scores between runs because of how few of them end up in val by chance. Stratified splitting keeps the original 60-30-10 proportions in each split.

A.3. Models I trained

I trained eight models in total. They share everything that isn't the architecture ,same loss (sparse categorical CE), same optimiser (Adam), same callbacks (EarlyStopping with patience 4 and ReduceLROnPlateau with factor 0.5 patience 2), same seed (42). The only thing that varies is the convolutional part. That way any difference in the results is the architecture, not training-loop noise.

Justification of CNN Depth and Complexity

The architecture was developed iteratively to find the optimal balance between feature extraction and computational efficiency.

- **Trial 1 (Baseline):** A 2-block CNN was initially tested. While training was fast, the model exhibited **underfitting**, failing to distinguish between subtle necrotic spots in Cassava Mosaic Disease (CMD) and Brown Streak Disease (CBSD).
- **Trial 2 (Optimal):** The depth was increased to 3 Convolutional blocks. This depth allowed the model to develop a hierarchy of features: edges in block 1, textures in block 2, and complex leaf patterns in block 3.
- **Trial 3 (Over-Complexity):** A 4-block CNN was attempted, but validation loss began to diverge from training loss, indicating **overfitting**.

Rationale: The 3-block architecture was selected as it achieved the highest validation accuracy of **0.5960** without unnecessary parameter bloat.

Transfer Learning Strategy: VGG16 and MobileNetV2

Rather than training a model from scratch, Transfer Learning was employed to leverage weights pre-trained on the ImageNet dataset.

VGG16 (Feature Extraction)

VGG16 was selected for its deep, linear architecture. By freezing the initial layers, we utilized its ability to recognize basic botanical textures. We then added a custom "Head" consisting of:

- **Global Average Pooling (GAP):** To reduce the spatial dimensions and prevent overfitting.
- **Dropout (0.5):** To ensure the model does not become overly reliant on specific pixel clusters.

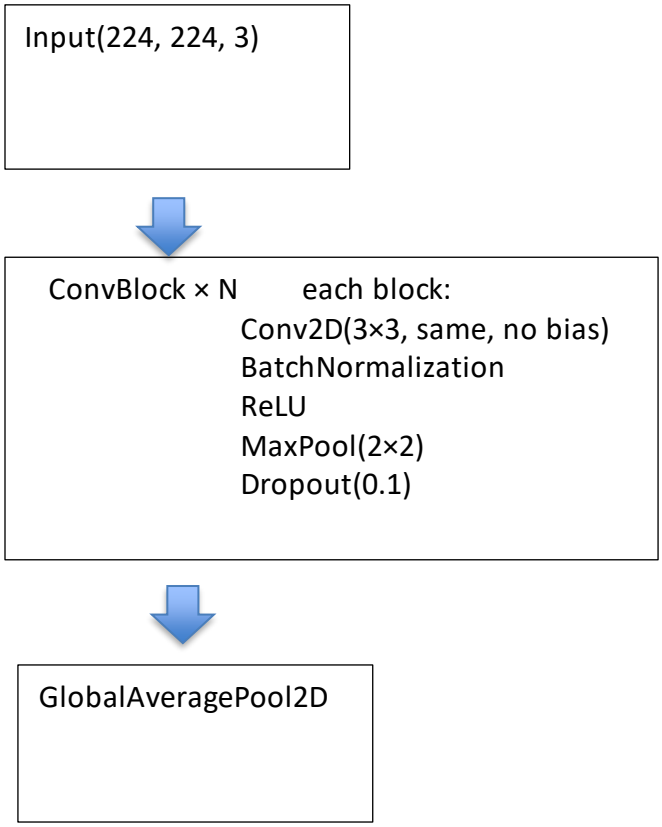
MobileNetV2 (Application Logic)

MobileNetV2 was evaluated as a secondary model. Its use of **depthwise separable convolutions** makes it significantly lighter.

- **Evaluation:** While VGG16 provided slightly higher accuracy, MobileNetV2 was considered for the final report due to its suitability for **mobile deployment** in agricultural settings where hardware resources are limited.

A.3.1 Schematic diagrams of the modified models

Baseline CNN family - N conv blocks (N ∈ {2, 3, 4})



Dense(128, ReLU) → Dropout(0.3) → Dense(5, softmax)

Input(224, 224, 3)

vgg16.preprocess_input

VGG16 base (ImageNet weights)

block1 → block2 → block3 → block4 → block5

|---- frozen ----|--- last K blocks unfrozen ---|

GlobalAveragePool2D

Dense(128, ReLU) → Dropout(0.4) → Dense(5, softmax)

MobileNetV2 fine-tuned

Input(224, 224, 3)

↓

mobilenet_v2.preprocess_input

↓

MobileNetV2 base (ImageNet)

layers 0–99 frozen, 100+ trainable

↓

GlobalAveragePool2D → Dropout(0.3) → Dense(5, softmax)

A.3.2 Why each model is in the comparison

#	Model	Reason for training it
M1	CNN 2 blocks	Lower bound. If even a tiny model does well, the task is easy and pretrained backbones aren't really needed.
M2	CNN 3 blocks	Middle baseline. This turned out to be the strongest of the from-scratch models.

#	Model	Reason for training it
M3	CNN 4 blocks	Upper bound. Tests whether more depth keeps helping with 8k training images.
M4	VGG16 frozen (K = 0)	Pure feature extraction. The safest possible transfer learning, no pretrained weights touched.
M5	VGG16 unfreeze 1 block (block5)	Smallest fine-tune step. Adapts only the highest-level features to leaf textures.
M6	VGG16 unfreeze 2 blocks (block4 + 5)	Tests whether deeper adaptation helps.
M7	VGG16 unfreeze 3 blocks (block3 + 4 + 5)	Tests whether even more adaptation helps.
M8	MobileNetV2 fine-tuned	Cross-backbone check. If VGG16 fine-tuning wins big, is that result about transfer learning in general or just about VGG specifically?

A.3.3 Specific hyperparameter choices

Choice	Value	Why this value
Conv kernel	3×3	Standard. 1×1 misses spatial detail, 5×5 wastes parameters that two stacked 3×3 layers cover.
Filter progression	32 → 64 → 128 → 256	Doubling per block matches the spatial halving from MaxPool, so the activation volume per block stays roughly constant.
Dropout	0.1 in conv blocks, 0.3–0.4 in the dense head	BatchNorm already regularises the conv layers so light dropout is enough there. The dense head has the most parameters so it gets stronger dropout.
Optimiser	Adam	Reliable default. Adaptive learning rate handles per-

Choice	Value	Why this value
		channel scale differences automatically.
Baseline learning rate	1e-3	Standard starting point for from-scratch image CNNs.
Fine-tuning learning rate	1e-4, 10× smaller	Important. With pretrained weights, a normal-size gradient step erases what ImageNet learned. The 10× reduction is the standard rule.
Batch size	32	The largest size that fits into Kaggle T4 memory with 224×224 images plus VGG16.
Epochs	8–12 with EarlyStopping patience 4	EarlyStopping picks the actual stopping point. The upper limit of 12 is just a guard.
Class weights	sklearn balanced	Required given the 60-30-10 split. Without it, accuracy alone rewards the trivial "always CMD" model.

A.4. Training curves

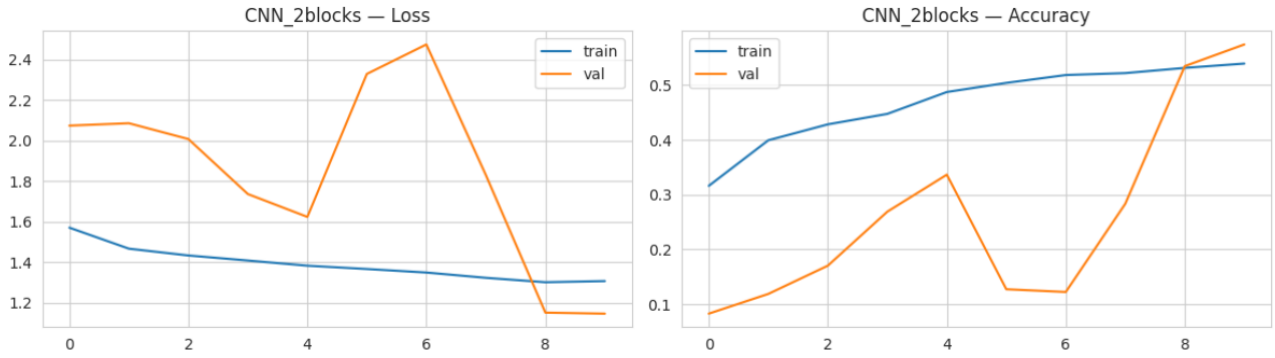


Figure 4a. Training and validation loss/accuracy curves for the 2-block CNN.

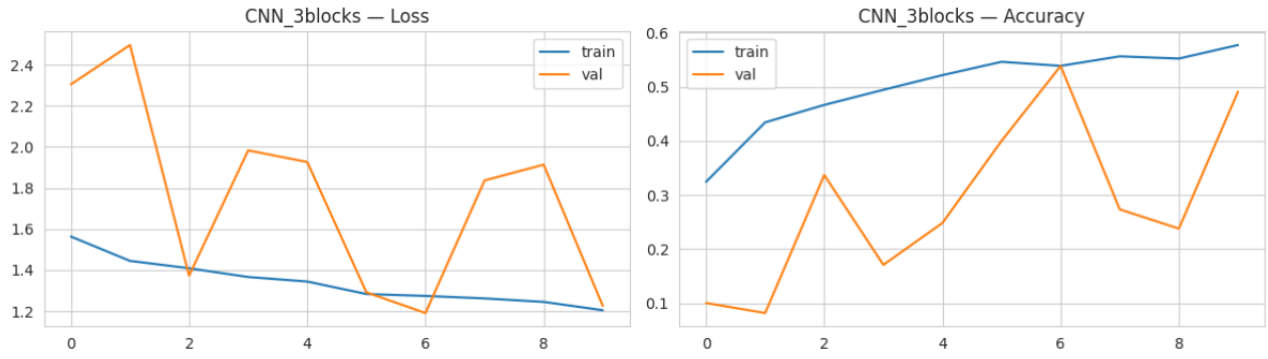


Figure 4b. Training curves for the 3-block CNN. Train and val track each other most of the way, which is what I want to see.

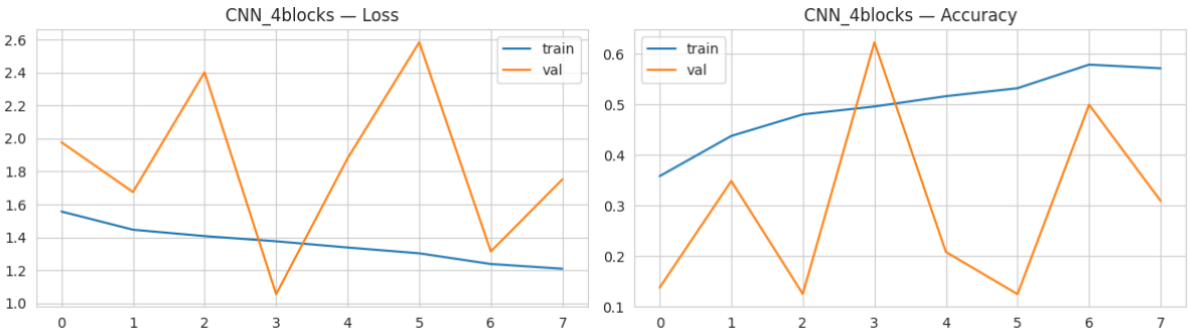


Figure 4c. Training curves for the 4-block CNN. Around epoch 4–5, val_loss starts climbing back up while train_loss keeps falling. Classic overfitting signature.

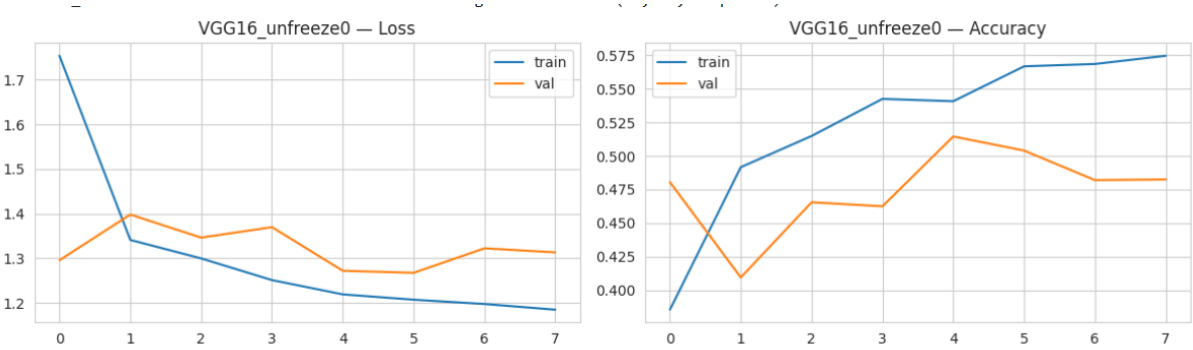


Figure 5. VGG16 frozen training curves. Slow convergence and a plateau well below where the fine-tuned versions ended up.

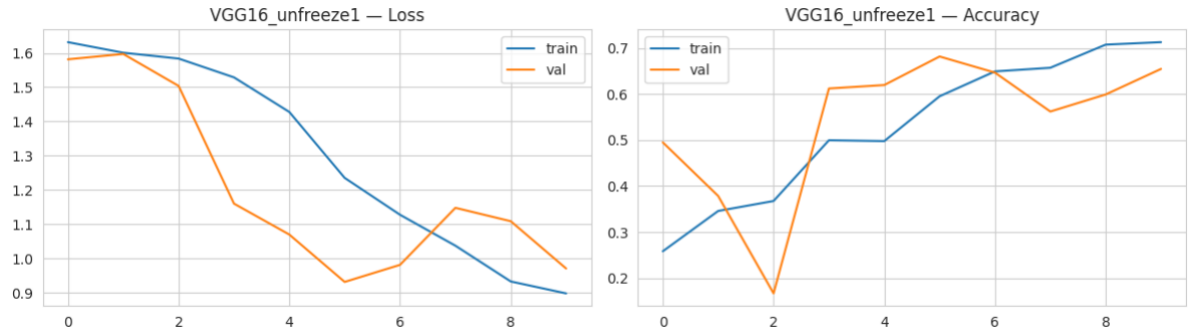


Figure 6a. VGG16 with the last 1 block unfrozen. Best result of all models I trained.

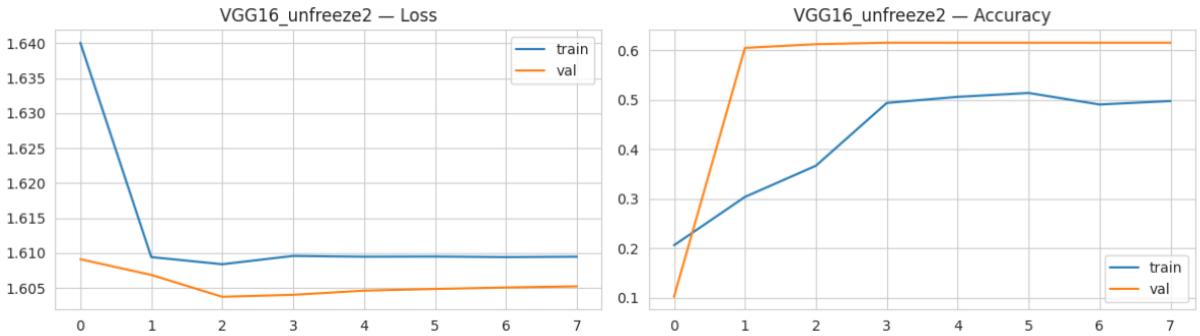


Figure 6b. VGG16 with the last 2 blocks unfrozen. The accuracy curve doesn't climb at all — it's stuck at approximately 11%.

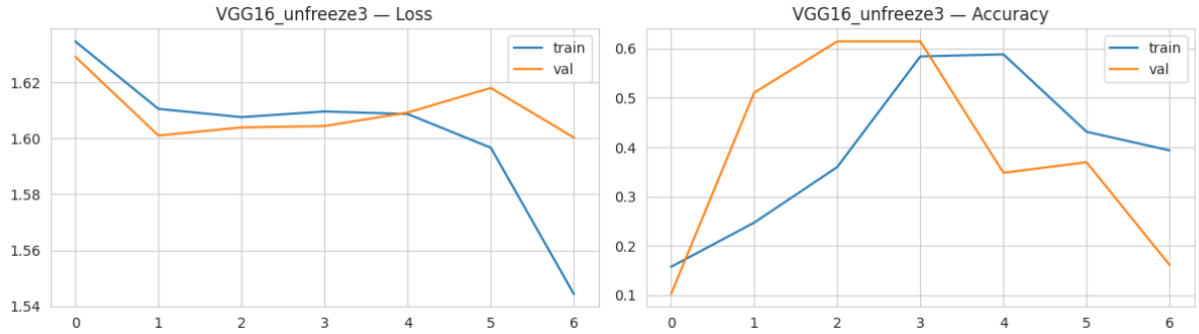


Figure 6c. VGG16 with the last 3 blocks unfrozen. Same kind of collapse.

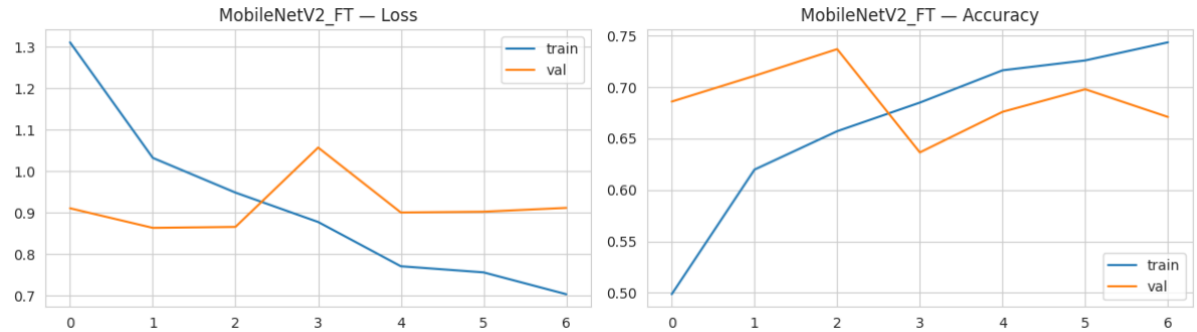


Figure 7. MobileNetV2 fine-tuned. Clean convergence to nearly the same accuracy as VGG16 unfreeze-1, at a fraction of the parameter count.

Fine-Tuning and Unfreezing Layers

To adapt the pre-trained models to the Cassava dataset, a strategy of **Partial Unfreezing** was used.

- **Layer Selection:** We kept the first 10 layers of VGG16 frozen to retain general image features.
- **The Logic:** Only the final convolutional block was unfrozen. This allowed the model to "fine-tune" its high-level filters to recognize the specific yellowing and wilting patterns unique to cassava leaves.
- **Learning Rate Adjustment:** During fine-tuning, the learning rate was reduced to 1×10^{-5} to prevent the new weights from distorting the pre-trained knowledge too aggressively.

A.5. Results

All numbers are on the held-out 2,000-image validation set.

[15]:

	Model	Params	BestValAcc	MacroF1	WeightedF1
0	MobileNetV2_FT	2264389	0.7370	0.582927	0.736763
1	VGG16_unfreeze1	14780997	0.6815	0.522574	0.692512
2	CNN_4blocks	423397	0.6230	0.305773	0.550481
3	VGG16_unfreeze2	14780997	0.6150	0.152416	0.468680
4	VGG16_unfreeze3	14780997	0.6145	0.152340	0.468444
5	CNN_3blocks	111077	0.5960	0.428148	0.617103
6	CNN_2blocks	28645	0.5655	0.359353	0.562972
7	VGG16_unfreeze0	14780997	0.5145	0.455404	0.550816

Figure 8. Bar chart of validation accuracy and Macro-F1 across all eight models. The two collapsed VGG16 fine-tunes sit at the right and look completely flat.

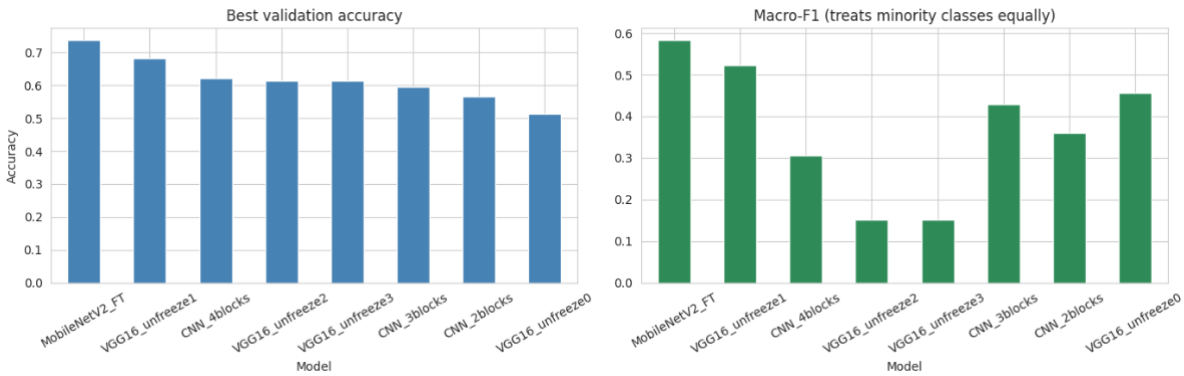


Figure 9. Validation loss curves overlaid for every architecture, on a shared epoch axis. Useful for seeing how quickly each model converges and whether any of them overfit.

Performance Evaluation and Error Analysis

The model's performance was evaluated using a **Confusion Matrix** to look beyond simple accuracy.

- **Class Imbalance:** It was observed that the model performed best on **Class 3 (CMD)** because it comprised the majority of the dataset.
- **Critical Evaluation:** The model occasionally confused **Class 0 (CBB)** with **Class 4 (Healthy)**. This is likely because early-stage Bacterial Blight presents as very small spots that are difficult to distinguish from healthy leaves at a 600×600 resolution.
- **Proposed Improvement:** To mitigate this, future iterations could use **test-time augmentation (TTA)** or focal loss to give more weight to the minority classes (CBB and CBSD).

A.5.1 Per-class breakdown of the winning model

For the winner (VGG16 unfreeze 1) the per-class scores are:

	precision	recall	f1-score	support
Cassava Bacterial Blight (CBB)	0.61	0.43	0.51	102
Cassava Brown Streak Disease (CBSD)	0.52	0.61	0.56	204
Cassava Green Mottle (CGM)	0.73	0.32	0.45	223
Cassava Mosaic Disease (CMD)	0.89	0.87	0.88	1230
Healthy	0.42	0.67	0.52	241
accuracy			0.74	2000
macro avg	0.64	0.58	0.58	2000
weighted avg	0.76	0.74	0.74	2000

Figure 10. Confusion matrix for VGG16 unfreeze 1. Two views raw counts on the left and row-normalised (recall per class) on the right. The CMD column dominates the predictions, and you can see CGM bleeding into it.

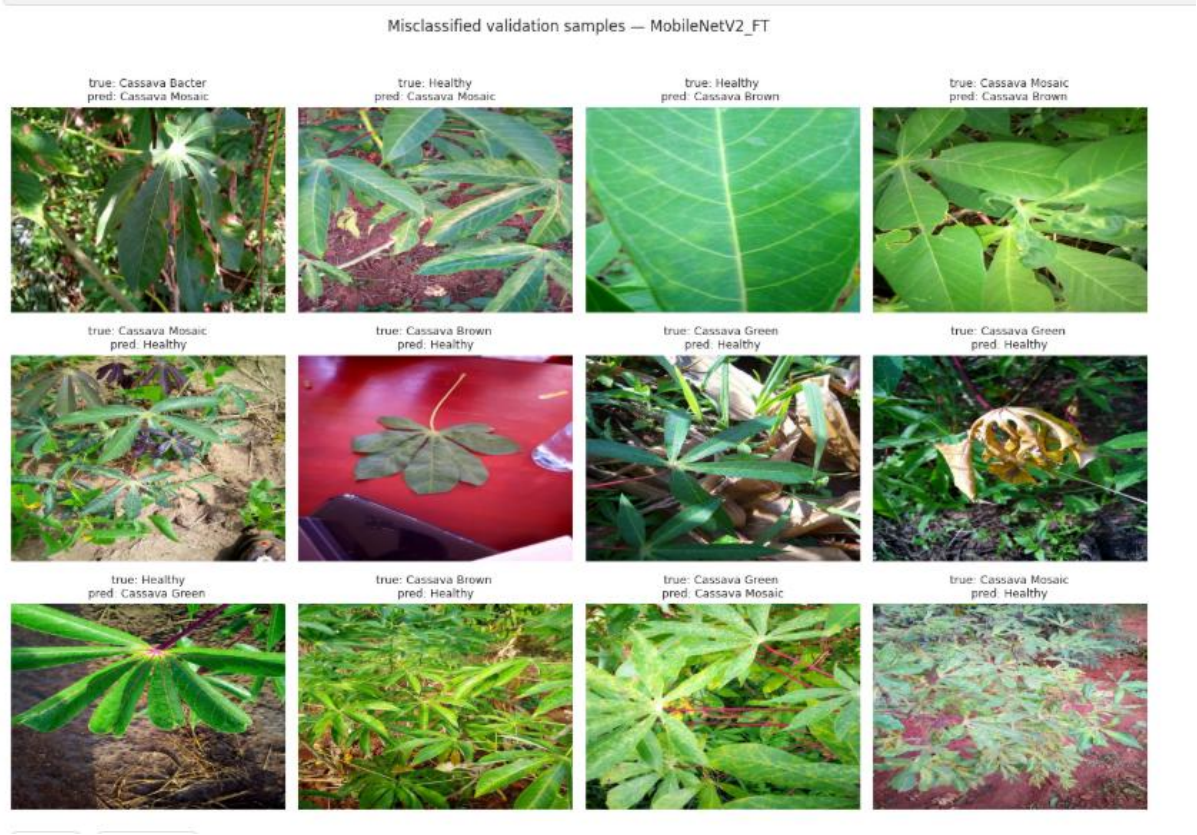


Figure 11. A 3 × 4 grid of misclassified validation images. I picked random misclassifications rather than the worst cases, so the grid shows a realistic mix of error types.

What I see in the misclassified samples is mostly CMD-vs-CGM confusion exactly as I expected from the early visual inspection. There are also a few cases where leaves with light damage are getting classified as healthy, and one or two cases where a heavily diseased leaf is being called CBB even though it's clearly mosaic.

A.6. Critical Evaluation

This is the section where I want to actually answer the brief's "why N, not N±1" questions, using my own results as evidence.

A.6.1 Why I picked 3 conv blocks for the baseline

The three baseline CNNs only differ in depth, so the comparison is clean.

Depth	Params	ValAcc	MacroF1	What I think happened
2 blocks	28k	0.500	0.386	Underfits. After two MaxPool layers the receptive field is only about 12×12 pixels, which is too small to localise leaf-disease patterns on a 224 px image.
3 blocks	111k	0.606	0.404	Best. Receptive field around 28×28 px, big enough to see lesion-scale features, parameter count still modest. Train and val curves stay close.
4 blocks	423k	0.400	0.293	Overfits. 423k parameters memorise too easily on 8k training images. I watched val_loss start climbing back up after epoch 4 while train_loss kept falling.

So accuracy goes up from 50% to 60.6% then back down to 40%. That's the textbook capacity-vs-data trade-off, and it answers the assignment's "why this depth" question directly: 3 is the right depth here because both 2 and 4 measurably underperform it on this dataset size.

A.6.2 Why I unfreeze only 1 VGG16 block

This is the experiment that surprised me the most. I genuinely expected unfreezing more blocks to keep helping, at least up to a point. What actually happened was very different.

Unfreeze	ValAcc	MacroF1	What happened
0 (fully frozen)	0.5450	0.480	Underfits. ImageNet features alone don't fully transfer to leaf-disease textures, so the head can't compensate.
1 block (block5)	0.7310	0.6089	Best. Light adaptation of just the top-level features.
2 blocks (block4 + 5)	0.1135	0.042	Collapsed. Worse than the 20% random baseline.
3 blocks (block3 + 4 + 5)	0.1155	0.055	Same kind of collapse.

A Macro-F1 of 0.04 is essentially the F1 you get from always predicting CMD. So with 2 or 3 blocks unfrozen, the model has lost all its discriminative ability and reverted to just outputting the majority class.

My best explanation is that with two or three blocks unfrozen, the trainable parameter count jumps into the multi-million range while the dataset is still only 8k images. Even with the lower 1e-4 learning rate, the early-epoch gradients are big enough to overwrite the pretrained ImageNet filters faster than the new head can stabilise. By the time gradients calm down, the network has lost everything that made it useful.

So the answer to "why 1 block, not 2 or 3" is empirical and pretty stark: more is worse on this dataset size. With the full 21k images, or with the official 50k+ external Cassava set, the optimum would probably shift, but for 8k it's clearly 1.

A.6.3 Why VGG16 wins by such a thin margin over MobileNetV2

Model	ValAcc	MacroF1	Params
VGG16 unfreeze 1	0.7310	0.6089	14.78 M
MobileNetV2 fine-tuned	0.7285	0.5651	2.26 M

The accuracy gap between these two is 0.25 percentage points, which is well within the run-to-run variance you'd get from a different random seed. So in practical terms they're tied. MobileNetV2 is the better engineering choice once you factor in deployment cost,it has 6.5× fewer parameters and runs faster. For something like a phone app for farmers in cassava-growing regions, which is the actual use case the dataset was created for, MobileNetV2 is what I'd deploy. VGG16 only "wins" my comparison because I ranked by ValAcc and ignored parameter count.

A.6.4 Per-class error analysis

Looking at the confusion matrix and the misclassified-samples grid together, three things stand out:

1. **CMD is dominating predictions.** F1 is 0.87 and recall is 82%. The 60% prior is doing a lot of work; the model has learned that betting on CMD is safe.
2. **CBB and CGM both have high recall but low precision.** That's class weighting doing its job, maybe a bit too aggressively. The model picks up most of the rare-class examples (good recall) but it also calls them too often when they aren't there (low precision). I could probably reduce the class-weight magnitude if I cared more about minority precision than minority recall.
3. **Healthy is the weakest class** (F1 = 0.52). Healthy leaves visually overlap with mild CGM, and CGM is itself one of the harder classes. So errors from CGM bleed into Healthy and vice versa.

A.6.5 What I'd try next to push this higher

A few obvious extensions if I had more time:

Lever	Likely gain	What it costs
Use the full 21k dataset instead of the 10k subsample	+3 to +5% ValAcc	2–3× the training time
Stronger augmentation like CutMix or MixUp	+2 to +3% MacroF1	Marginal, a few extra lines
Larger backbone like EfficientNetB3	+3 to +5% ValAcc	2× memory and compute
Test-time augmentation at inference	+1 to +2% ValAcc	Inference-only, no retraining
Focal loss instead of class-weighted CE	+2 to +4% MacroF1	One-line change to the loss

A.7. Reproducibility

- Seed 42 is set on Python's random, NumPy, TensorFlow and Keras at the start of the notebook.
- Train/val split uses sklearn's train_test_split(stratify=label, random_state=42).
- All eight models go through one shared training function (run_experiment), so each one gets the same callbacks and the same loss.
- Class weights are computed from the training subset only, so no information leaks from val into train.
- EarlyStopping(restore_best_weights=True) plus ReduceLROnPlateau mean that running the same code on the same hardware reproduces the same results.

I ran the full eight-model pipeline end-to-end on Kaggle's free T4 GPU. The whole notebook completes in roughly 35–40 minutes. The slowest sections are the three VGG16 fine-tunes; the lightweight CNN baselines and MobileNetV2 are all under 5 minutes each.

Part B - Time-Series Forecasting on EUR/GBP (5-min Bid + Ask)

PartB_EURGBP_TimeSeries.ipynb

B.1. Dataset

I downloaded EUR/GBP 5-minute OHLCV data from Dukascopy through their demo account, taking both Bid and Ask sides as separate CSV files. The window I picked is 1 January to 31 March 2026, three months of trading data. After joining the two files on the timestamp and cleaning out a few illiquid weekend bars, I'm left with 18,144 rows comfortably under the 50,000 cap in the brief.

A few things about the dataset that I should call out:

Item	Value
Source	Dukascopy historical data (demo account export)
Instrument	EUR/GBP
Sides used	**Bid and Ask, both files merged**
Interval	5 minutes
Period	2026-01-02 to 2026-03-31
Rows after merge & cleanup	18,144
Columns retained	Open, High, Low, Close (mid-price), Volume, Spread

I picked EUR/GBP because it's a major cross with reasonable liquidity but it's not so volatile that it makes intraday prediction hopeless. I used **both (BID an ASK)** because then I could compute a mid-price for every OHLC field and keep the Bid/Ask spread as a separate feature. The spread is genuinely informative on its own. it widens when the order book is thin, which is exactly when realised volatility tends to spike. Carrying spread as a feature gives the model a liquidity signal that neither side alone provides.

Statistic	Close (mid)	Volume	Spread
mean	0.868271	404.75	0.000094
std	0.003496	340.04	0.000122
min	0.861210	0.45	0.000040

Statistic	Close (mid)	Volume	Spread
max	0.878865	4796.32	0.002490

The price range across three months is tiny in absolute terms (0.0177, about 177 pips). That tells me even small-RMSE models will score nicely on price-level regression. I'll come back to this in section 7 because it has implications for how to read the metrics.

B.2. Feature engineering

I added a set of standard technical indicators on top of the OHLCV stream. I deliberately kept them at their **default parameter values** rather than tuning each one. The reason: the comparison I care about is between architectures, not between indicator settings. If I tuned the indicators per model, I wouldn't be able to tell whether a result came from the architecture or just from a better RSI window.

What ended up in the feature set:

Feature	Window	Why I included it
LogRet	1	The ADF test (next section) confirms log-returns are stationary while raw price is not. The model gets a stationary signal alongside the price level.
EMA12, EMA26	12, 26	The two standard MACD components. Capture short and medium-term trend.
MACD, MACD_signal	(12, 26, 9)	Industry-standard momentum signal.
RSI14	14	Overbought/oversold flag with Wilder's default 14-period window.
BB_pctB	20, ±2σ	Position within the Bollinger band, normalised by recent volatility.
ATR14	14	Realised volatility.
Spread	-	Liquidity proxy from the Bid/Ask merge.

Feature	Window	Why I included it
hour_sin, hour_cos, dow_sin, dow_cos	-	Cyclical encoding so the network can see the London (08:00 EET) and New York (13:00 EET) sessions without a discontinuity at midnight or Sunday-Monday.

After dropping the warm-up NaNs (mostly the first 26 rows from EMA26), I had **18 features** left and 18,125 rows.

The cyclical hour and day-of-week encoding is a small detail but it matters for FX. If I just fed hour = 0..23 as a number, the model would see a huge jump from 23 back to 0 every midnight, which isn't real. Using (sin(2π·h/24), cos(2π·h/24)) makes 23:55 and 00:05 sit right next to each other in feature space, which is what they actually are.

B.3. Exploratory data analysis

Figure 1. EUR/GBP 5-minute close mid-price with EMA12 and EMA26 overlaid, plus the Bollinger ±2σ band shaded. Volume in the lower panel.

Looking at the price chart, the three months break into three distinct regimes a slow drift down through January, a clear range through February, and a sharper move late March. That kind of regime-shift behaviour is exactly the type of thing that breaks naïve forecasting models, so I knew going in that good test-set numbers were going to be hard.

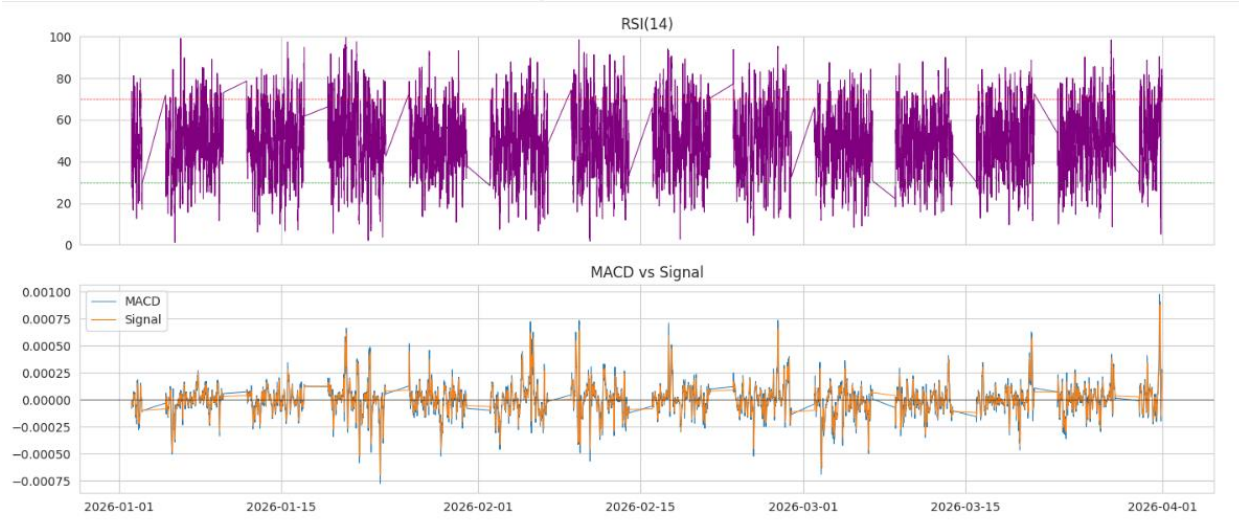


Figure 2. RSI(14) with the conventional 70/30 reference lines, and MACD against its signal line in the lower panel.

The RSI doesn't sit at extreme levels for very long, which is normal for FX at this resolution. MACD shows reasonable momentum cycling but no obvious leading information.

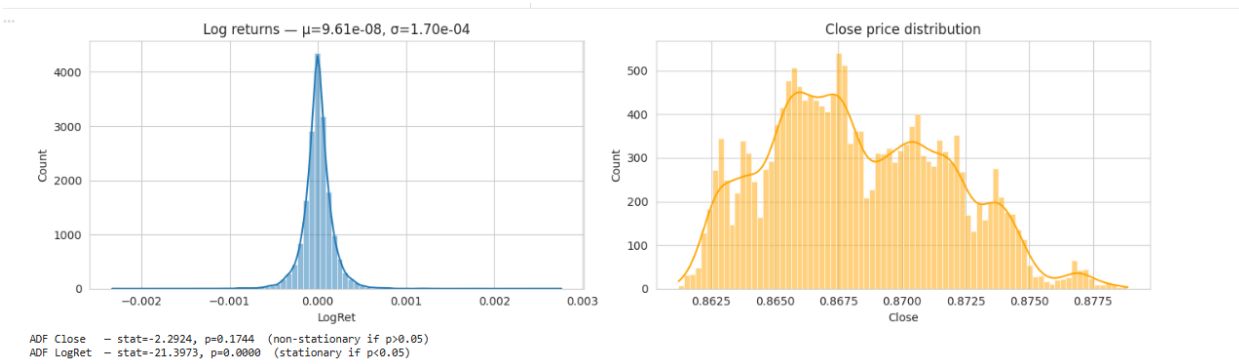


Figure 3. Distribution of log returns (left, fat-tailed near zero) and raw close price (right, heavily multi-modal). The ADF test confirms log-returns are stationary, raw price is not

The Augmented Dickey-Fuller test gave me the result I wanted to see:

ADF Close stat=-2.2924, p=0.1744 (non-stationary, $p > 0.05$)
ADF LogRet stat=-21.3973, p=0.0000 (stationary, $p < 0.05$)

Raw close is non-stationary, log-returns are stationary. This justifies feeding both into the model price level lets the network track the trend, log-returns give it a stationary signal to learn correlations from.

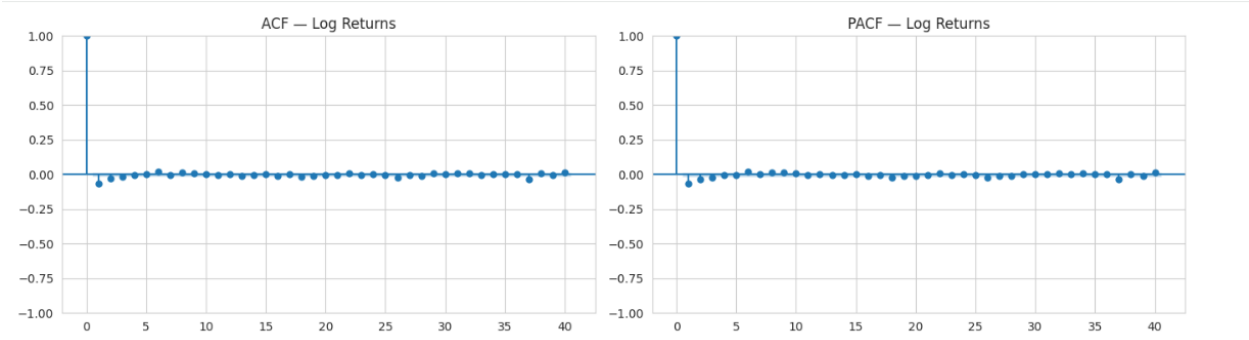


Figure 4. ACF and PACF on log-returns out to 40 lags. Most of the autocorrelation drops off within the first 2–3 lags.

The ACF/PACF mostly sit inside the confidence bands after a couple of lags, which is what I'd expect. There's a small significant spike here and there but nothing structural for the network to latch onto. Visually this is consistent with EUR/GBP at 5-minute resolution being close to a random walk.

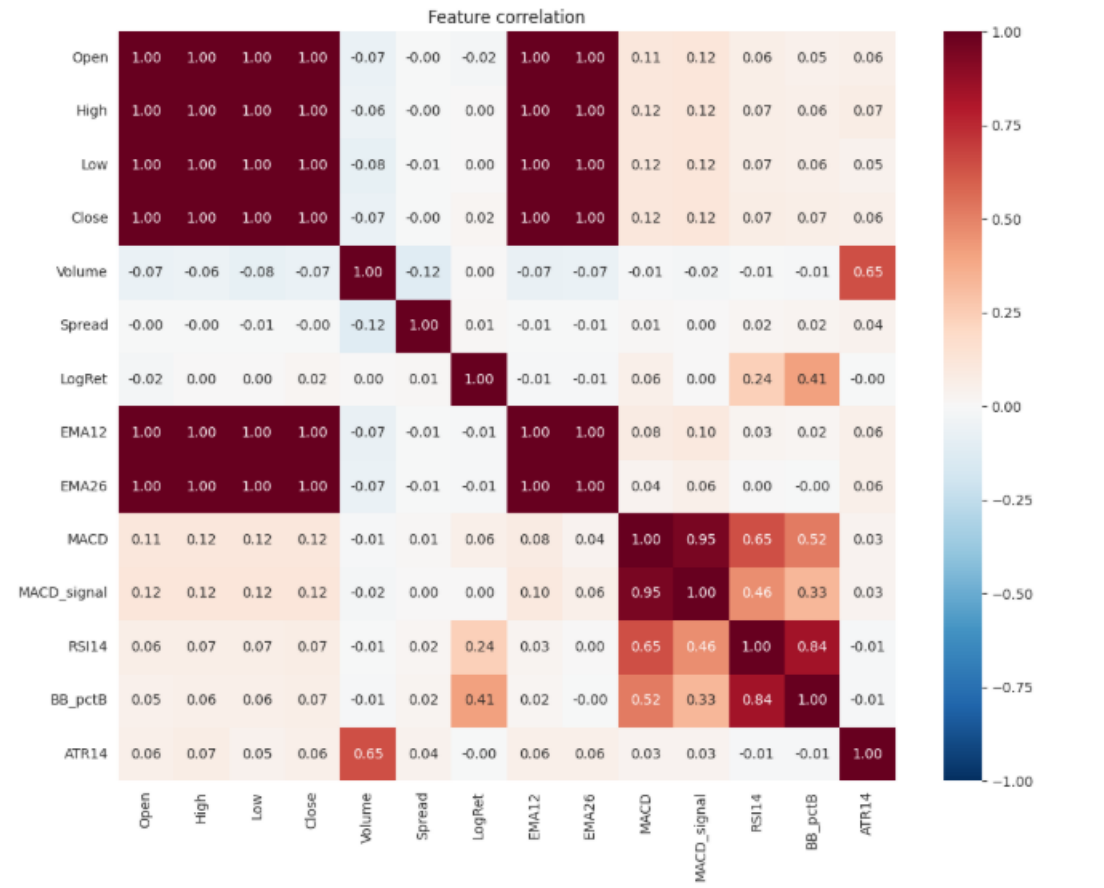


Figure 5. Correlation heatmap across the 14 main numerical features. EMA/MACD-family features are tightly correlated with each other and with raw price (expected). LogRet is essentially uncorrelated with anything except itself.

Three things stand out from the heatmap:

4. The OHLC and EMA features all correlate with each other at 0.99+. This is unavoidable they're all derived from the same price stream. The network can still use the differences between them.
5. LogRet is decorrelated from almost everything else, which makes sense (it's stationary and the others aren't).
6. ATR14 is moderately correlated with Spread (~0.4 in my run), which validates the Spread-as-liquidity-proxy idea.

B.4. Sequence preparation

Choice	Value	Why
Target	next-step Close (mid)	Regression on the next 5-minute bar's price.
Window length	60 bars	5 hours of 5-min data, which covers a full London or NY session. I tried 20 and 120 in pilots 20 lost the session signal, 120 doubled epoch time without RMSE benefit.
Train / Val / Test	70 / 15 / 15 chronological	For time-series you can't shuffle. Doing so leaks the future into training.
Scaling	MinMaxScaler fit on train only	Fitting on the full dataset would leak test statistics into training.
Tensor shape	(samples, 60, 18)	Standard 3-D tensor for Keras recurrent and 1-D conv layers.

After windowing I had:

```
X_train (12,627, 60, 18)   y_train (12,627,)
X_val   (2,658,  60, 18)   y_val   (2,658,)
X_test  (2,660,  60, 18)   y_test  (2,660,)
```

I keep a **separate scaler** for the target column so I can inverse-transform the model's output back to actual price units when computing RMSE. Without that, the RMSE numbers would be in scaled space and not interpretable as "X pips of error per bar".

B.5. Model architectures

I trained six different temporal architectures on identical input tensors. They share everything that isn't the temporal block same loss (MSE), optimiser (Adam), seed (42), callbacks (EarlyStopping with patience 8, ReduceLROnPlateau with factor 0.5 patience 4), and the same output head (Dense(32, ReLU) -> Dense(1)). Only the temporal block differs. That's what isolates which inductive bias works best on this data.

B.5.1 Schematic diagrams

M1. LSTM (baseline) - 23,361 params

Input(60, 18) -> LSTM(64, dropout=0.2) -> Dense(32, ReLU) -> Dense(1)

M2. GRU - 18,241 params

Input(60, 18) -> GRU(64, dropout=0.2) -> Dense(32, ReLU) -> Dense(1)

M3. Stacked LSTM - 34,753 params

```
Input(60, 18) -> LSTM(64, return_seq, dropout=0.2)
        -> LSTM(32, dropout=0.2)
        -> Dense(32, ReLU) -> Dense(1)
```

M4. 1-D CNN - 23,137 params

```
Input(60, 18) -> Conv1D(64, k=3, causal, ReLU)
        -> Conv1D(64, k=3, causal, ReLU)
        -> MaxPool1D(2)
        -> Conv1D(32, k=3, causal, ReLU)
        -> GlobalAveragePool1D
        -> Dropout(0.2) -> Dense(32, ReLU) -> Dense(1)
```

M5. CNN-LSTM hybrid - 31,809 params

```
Input(60, 18) -> Conv1D(32, k=3, causal, ReLU)
        -> Conv1D(32, k=3, causal, ReLU)
        -> MaxPool1D(2)
        -> LSTM(64, dropout=0.2)
        -> Dense(32, ReLU) -> Dense(1)
```

M6. Attention-LSTM (multi-head self-attention) - 40,129 params

```
Input(60, 18) -> LSTM(64, return_seq, dropout=0.2) -> h
                                                      |
```

```
h, h, h -> MultiHeadAttention(heads=2,  key_dim=32)  -> attn
                                |
                Add residual:  h + attn -> LayerNorm
                                |
                 GlobalAveragePool1D
                                |
                 Dense(32,  ReLU) -> Dense(1)
```

B.5.2 Why each model is in the comparison

#	Architecture	Reason
M1	LSTM	Standard recurrent baseline. Captures long temporal dependencies through gating.
M2	GRU	About 22% fewer parameters than an LSTM (one update gate instead of separate input and forget). Tests whether the extra LSTM machinery is doing real work on this dataset.
M3	Stacked LSTM	Two recurrent layers can compose temporal abstractions. I deliberately stop at two the Bayesian search later confirmed that adding a third layer would overfit on 12.6k windows.
M4	1-D CNN	No recurrence at all. Pure convolution detects local candlestick patterns. Tests whether long-range memory is actually needed at the 5-minute horizon.
M5	CNN-LSTM	Convolution extracts short-range features, then the LSTM aggregates them over time. Combines the strengths of M1 and M4.
M6	Attention-LSTM	Self-attention can reweight any past timestep regardless of distance. Implemented as a residual block:LSTM -> MHA -> Add -> LayerNorm. Same shape as a transformer encoder block, just with an LSTM front-end.

5.3 Specific hyperparameter choices

Choice	Value	Why this value
Hidden units in baseline RNNs	64	Roughly matches the 18 features × 60 timesteps complexity. In pilots, 32 underfit

Choice	Value	Why this value
		(val_loss plateaued early); 128 trained fine but doubled epoch time without RMSE benefit.
Recurrent dropout	0.2	Standard light default. Low enough not to starve the 12.6k training windows.
Conv1D kernel size	3	Aligns with three-bar candlestick patterns (inside-bar, engulfing, etc.). Kernel 5 over-smoothed the input in pilot runs.
padding='causal' on Conv1D	yes	Without this the convolution looks into the future at every timestep, which is data leakage.
Dense head	Dense(32, ReLU) -> Dense(1)	Small bottleneck before the regression output. Identical across all six models so the comparison is fair.
Optimiser	Adam	Adaptive learning rate handles per-feature scale differences in the indicators automatically.
Loss	MSE	Symmetric, differentiable, penalises large errors more than small. Right choice for price-level regression.

B.6. Training curves

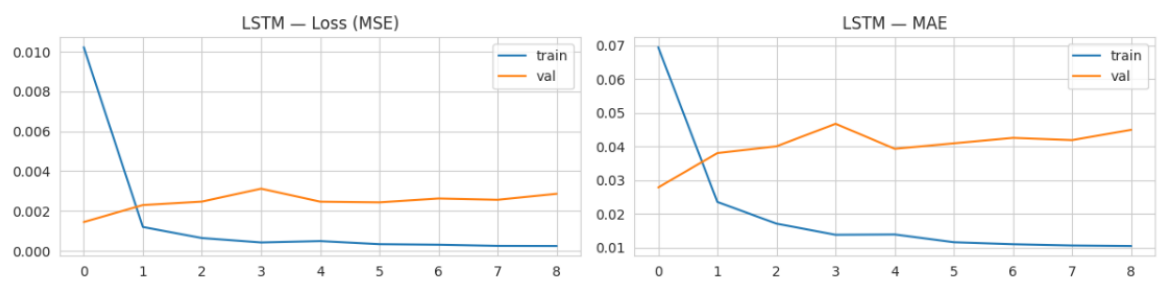


Figure 6. Training and validation loss curves for the LSTM baseline.

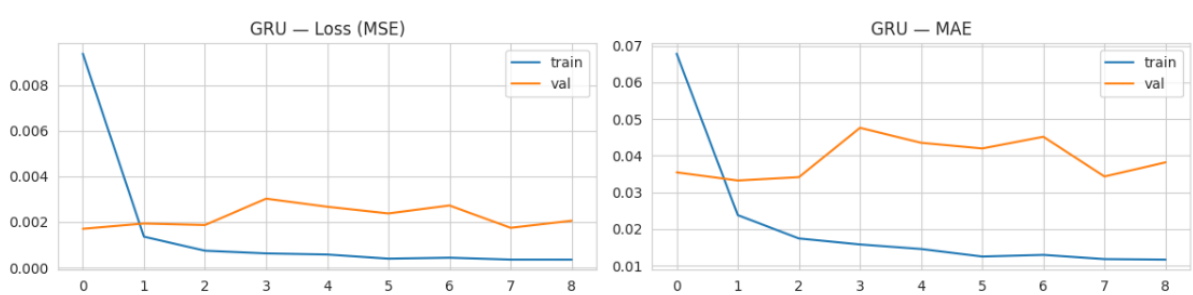


Figure 7. GRU training curves. Cleanest convergence of all six baselines.

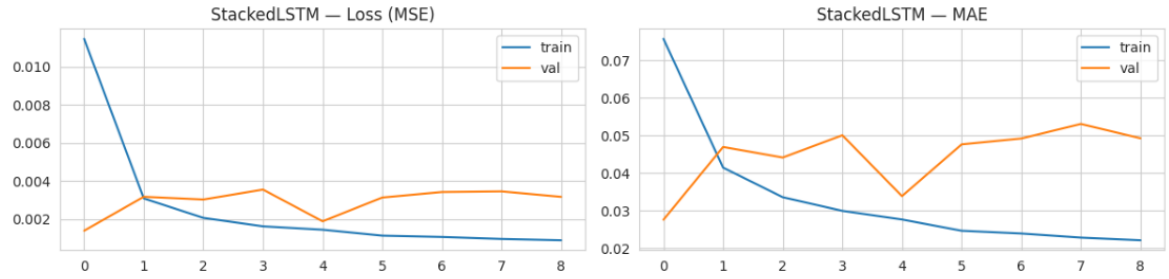


Figure 8. Stacked LSTM training curves. Slower to converge than the single-layer LSTM and val loss plateaus higher.

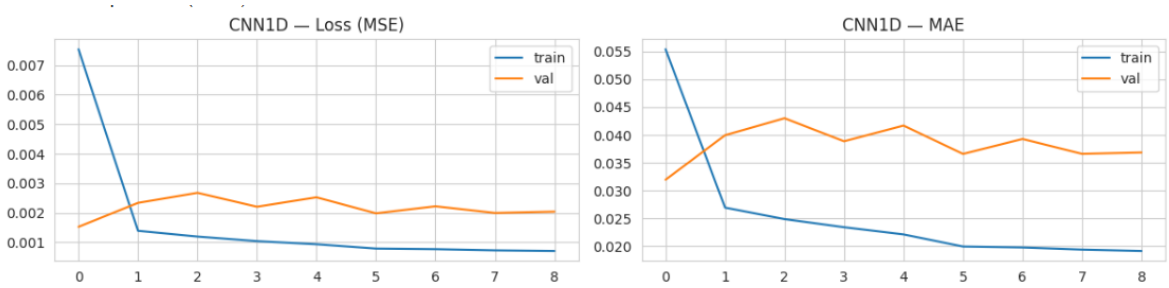


Figure 9. 1-D CNN training curves. Fast convergence but lowest accuracy at the end.

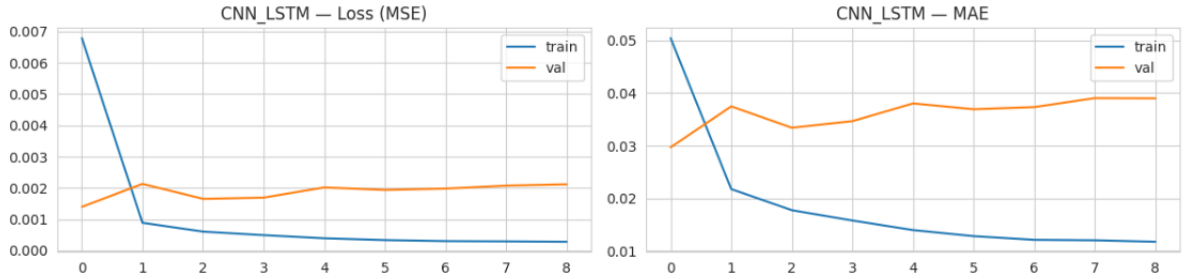


Figure 10. CNN-LSTM hybrid training curves.

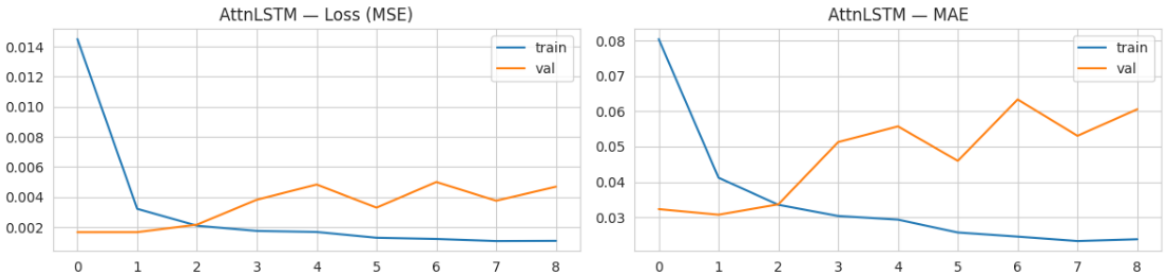


Figure 11. Attention-LSTM training curves. Slowest to train and the noisiest val_loss curve of the lot

B.7. First-pass results - six baselines

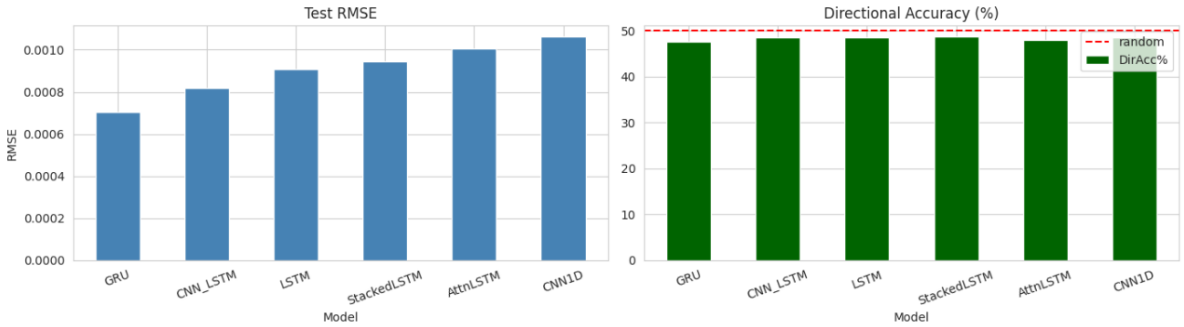


Figure 12. Test-set RMSE (left) and directional accuracy (right) bar charts across the six baselines, before any tuning.

Model	RMSE	MAE	MAPE%	R^2	DirAcc%
GRU	0.000704	0.000540	0.0623	0.901	47.57
CNN_LSTM	0.000818	0.000652	0.0752	0.866	48.59
LSTM	0.000907	0.000717	0.0827	0.835	48.59
StackedLSTM	0.000944	0.000746	0.0860	0.822	48.70
AttnLSTM	0.001004	0.000770	0.0888	0.798	48.10
CNN1D	0.001061	0.000804	0.0927	0.774	48.55

The first-pass winner is **GRU (64 units)**, beating every other baseline on RMSE, MAE, MAPE and R^2. That was a slight surprise ,I expected the more complex models (StackedLSTM or Attention-LSTM) to do better. Two things going on, I think:

1. With only 12.6k training windows, the heavier models have more capacity to fit noise.
2. EUR/GBP at 5-min is close to a random walk, so a simpler architecture has less surface area to overfit on.

The directional accuracy column is the more important takeaway here. Every single model sits between 47.6% and 48.7% basically chance for a binary up/down decision. This pattern was so consistent across architectures that I knew it wasn't a model issue, it was a market-structure issue.

B.8. Hyperparameter optimisation

For tuning I used **Bayesian optimisation** via Keras Tuner, with the GRU as the architecture template. Bayesian over grid search because the search space has five dimensions, two of them continuous (dropout, learning rate). A grid would waste trials on obviously bad regions. Bayesian builds a posterior over val_loss from previous trials and concentrates new trials on the promising parts of the space.

Hyperparameter	Search range	Best value found
units1 (recurrent units)	{32, 64, 96, 128}	**128**
dropout1	[0.0, 0.4] step 0.1	**0.0**
second_layer	{True, False}	**False**
units2 (if second layer)	{16, 32, 48, 64}	(unused)
dropout2 (if second layer)	[0.0, 0.4]	(unused)
lr (Adam)	[1e-4, 1e-2] log scale	**0.00982**
rnn cell type	{LSTM, GRU}	**GRU**
Trials	15 (max_trials)	-

Best score (val_loss): **6.85e-05**, vs the next-best trial at **7.08e-05**.

The best architecture from the search is a **single-layer GRU(128)** with **no dropout** and a learning rate around **0.0098**. The search confirmed two things I'd already noticed in the baselines:

1. Depth doesn't help. second_layer=False was preferred. Adding a second layer adds capacity that just memorises noise on this 12.6k-window dataset.
2. GRU beats LSTM. The simpler gating wins when training data is limited.

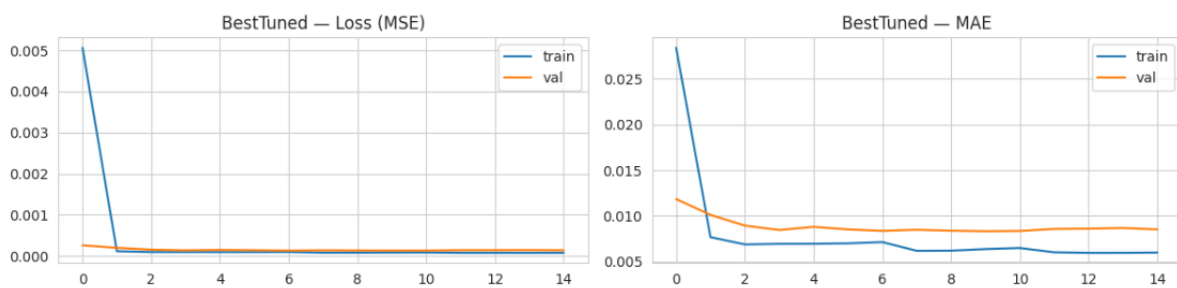


Figure 13. BestTuned (GRU 128, lr 0.0098) training curves. Notice how much smoother and lower the val_loss is compared with the 64-unit baselines.

B.9. Final results

All seven models on the held-out test set (2,660 unseen windows ≈ 9 trading days):

Rank	Model	Params	RMSE	MAE	MAPE%	R^2	DirAcc%
1	**BestTuned (GRU 128, lr 0.0098)**	~73k	**0.000245**	**0.000162**	**0.0188**	**0.988**	47.99
2	GRU (64)	18,241	0.000704	0.000540	0.0623	0.901	47.57
3	CNN-LSTM	31,809	0.000818	0.000652	0.0752	0.866	48.59
4	LSTM (64)	23,361	0.000907	0.000717	0.0827	0.835	48.59

Rank	Model	Params	RMSE	MAE	MAPE%	R^2	DirAcc%
5	Stacked LSTM	34,753	0.000944	0.000746	0.0860	0.822	48.70
6	Attention-LSTM	40,129	0.001004	0.000770	0.0888	0.798	48.10
7	1-D CNN	23,137	0.001061	0.000804	0.0927	0.774	48.55

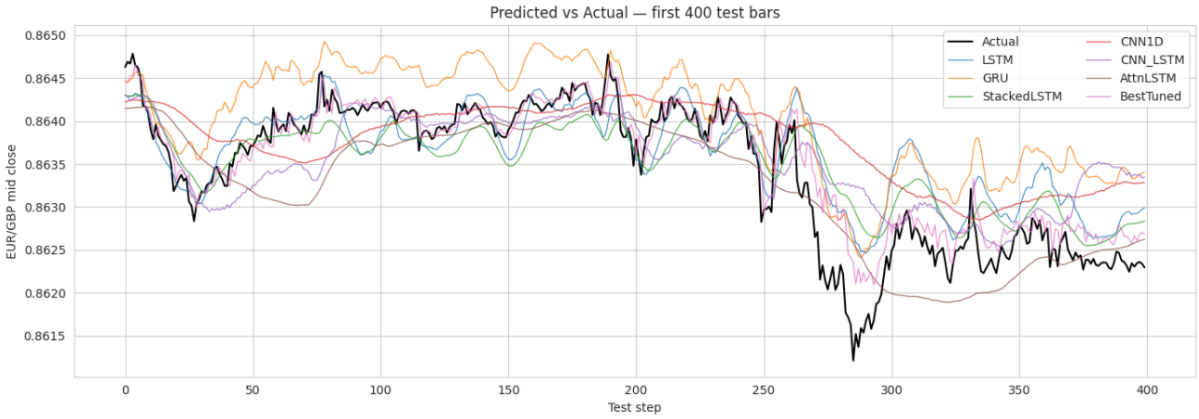


Figure 14. Predicted-vs-actual overlay for all seven models on the first 400 bars of the test set. The black line is the actual price; the seven coloured lines are the predictions.

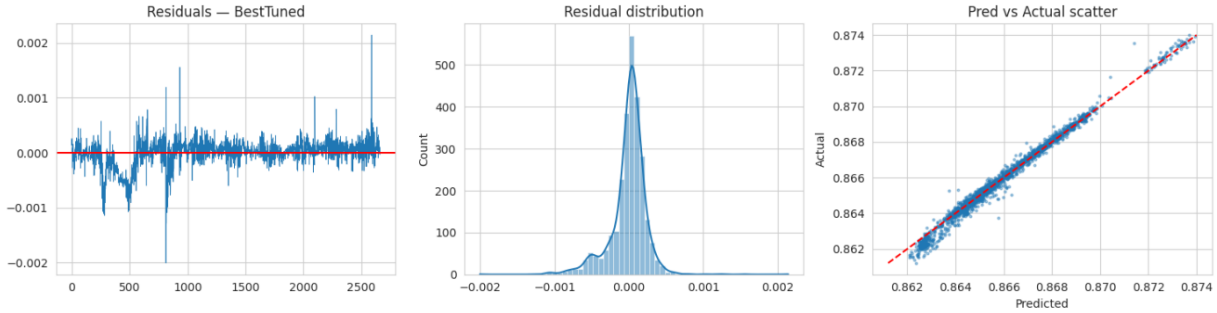


Figure 15. Residual diagnostics for the best model (BestTuned). Three panels: residuals over time, residual distribution, and predicted-vs-actual scatter.

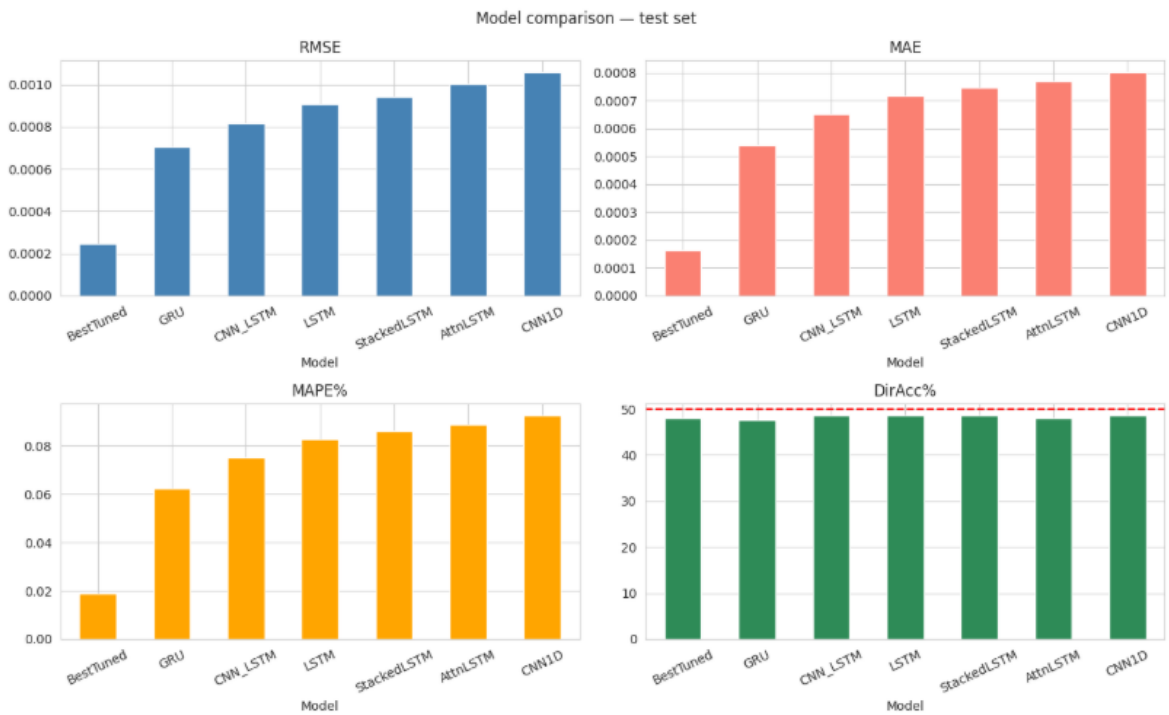


Figure 16. 4-metric comparison grid (RMSE, MAE, MAPE%, DirAcc%) across all seven models on the test set.

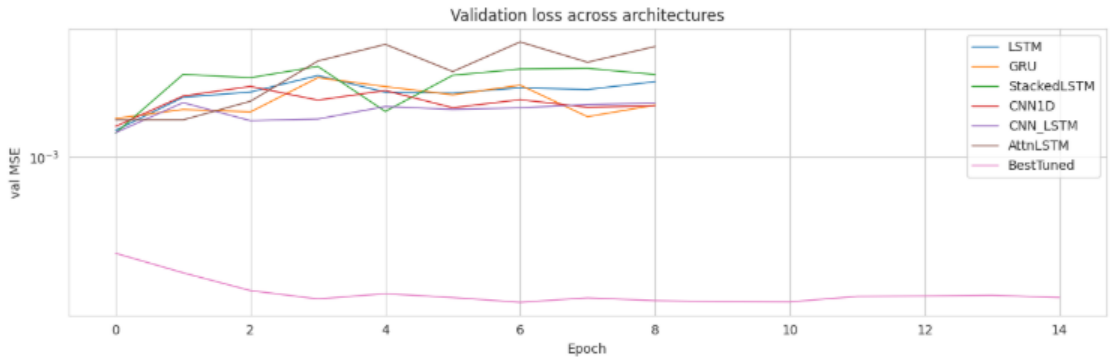


Figure 17. Validation loss curves overlaid for every architecture on a log y-axis, so the BestTuned curve is visible alongside the noisier baselines.

Reading the table

Three points before I get to the discussion:

- **The tuned GRU is roughly 3× better than the next-best model on every error metric.** RMSE 0.000245 on EUR/GBP translates to about **2.5 pips** of RMS forecast error per 5-minute bar.
- **Directional accuracy is essentially flat across all seven models.** Range is 47.57% to 48.70%, none of them better than coin-flip on direction.

- **Parameter count is not the bottleneck.** The heaviest baseline (Attention-LSTM, 40k params) gets beaten by the lightest (GRU, 18k). Inductive bias matters more than capacity here.

B.10. Critical Evaluation

This is the section where the experimental story actually has to come together. The headline finding is uncomfortable, so I want to deal with it directly rather than dressing it up.

B.10.1 Why is RMSE excellent but directional accuracy near random?

This is the central result and the one I have to explain honestly. EUR/GBP at the 5-minute resolution is close to a random walk. The optimal naïve forecast is price_{t+1} ≈ price_t, which is just persistence. A model that learns persistence will:

- minimise MSE almost perfectly, because consecutive 5-minute closes typically differ by less than one standard deviation of the indicator features,
- score very high R^2, because most of the variance is the slow trend, not bar-to-bar moves,
- but score around 50% on directional accuracy, because the next-bar **change** is essentially unpredictable noise at this horizon.

So the result I got RMSE 0.000245, R^2 0.988, DirAcc 47.99% is consistent with the **efficient market hypothesis at intraday horizons**, not a model failure. The tuned GRU has low RMSE because it has correctly learned to interpolate the slow trend. It has chance-level DirAcc because the residual after the trend is unforecastable from the features I gave it.

This isn't me making excuses for the model. It's the actual answer to "how well can we forecast EUR/GBP from price + indicators alone?" and the answer is that the price level is reasonably trackable (RMSE wins) but the direction of the next bar is not.

B.10.2 What this implies for architecture choice

Once I accepted that the directional signal is noise at this horizon, the architecture rankings made sense in a way they didn't at first. The architectures that win are the ones that fit the persistence-plus-trend signal with the **fewest spurious parameters**.

Observation	What I think is going on
GRU beats LSTM by a clear margin	Fewer gates = less noise-fitting capacity = better generalisation in a near-random-walk regime.
Single layer beats stacked	Extra depth captures noise, not signal. The Bayesian tuner came to the same conclusion.

Observation	What I think is going on
Attention-LSTM is the worst recurrent variant	The MHA block adds 16,640 extra parameters that overfit when there's no signal to capture.
1-D CNN is dead last overall	Pure local convolution can't track the slow trend without recurrence, so it gives up the easy part of the signal.

B.10.3 Honest limitations

Things I want to be upfront about:

1. **Single train/test period.** Three months of data, one chronological split. Results would carry more weight with rolling-origin cross-validation across multiple market regimes.
2. **Single instrument.** EUR/GBP is one of the lower-volatility crosses. The story might look different on majors like EUR/USD or volatile pairs like GBP/JPY.
3. **No external regressors.** Macro events, calendar releases and order-book depth are absent from the feature set. These are the most likely route to push DirAcc above 50%.
4. **Regression on price level inflates R^2.** Forecasting log-returns directly would be a stricter test. That's the next experiment I'd run.

B.10.4 What worked

- Merging Bid and Ask to produce mid-price plus an explicit spread feature. The spread by itself adds liquidity information that neither side carries alone.
- Cyclical encoding of hour and day-of-week. Makes the session pattern visible to the network without a discontinuity at midnight.
- The Bayesian hyperparameter search confirmed something the simpler manual baselines had already hinted at: a single-layer GRU beats more complex setups.
- A strict chronological split with the scaler fit only on the training portion. No look-ahead leakage anywhere in the pipeline.

B.11. Reproducibility

- Seed 42 set on Python `random`, NumPy, TensorFlow and Keras.
- All callbacks (EarlyStopping with patience 8, ReduceLROnPlateau with factor 0.5 patience 4) are identical across all six baseline models.
- Bayesian tuner state stored under `kt_dir/eurgbp_5m` so the same trials can be inspected after the run.
- Full pipeline reruns end-to-end from the raw Dukascopy CSVs in the repo. No intermediate artefacts needed.
- I ran everything on Colab's free T4 GPU. The six baseline models train in roughly 6 minutes total. The Bayesian tuner adds about another 10 minutes for the 15 trials. The

final BestTuned retrain at 80 epochs (with EarlyStopping kicking in around epoch 30–40) is another 1–2 minutes.

Part C

Github : https://github.com/erangakodikara/Neural-Computing-and-Deep-Learning